

DA6400; Programming Assignment - 03

Rohit Kumar (DA24S003)
Sankalp Shrivastava (DA24S021)

May 3, 2025

You can find the full implementation on GitHub repository.

1. Introduction

In this assignment, we implement and compare two algorithms proposed in the paper: **1-step SMDP Q-learning** and **Intra-Option Q-learning**, using a simplified domain known as the Taxi environment. The aim is to leverage temporally extended actions (options) to improve learning efficiency and performance over primitive, one-step actions. This work serves as a hands-on exploration of hierarchical learning concepts such as subgoals, option termination, and inter-option learning updates.

2. Environment Description

The environment used in this assignment is the **Taxi-v3** environment from the OpenAI Gym (now Gymnasium) library. It is a classic grid-world domain designed for benchmarking reinforcement learning algorithms. The environment consists of a **5×5 grid** representing a city map where a taxi must navigate to pick up and drop off a passenger at one of four designated locations marked as **Red (R)**, **Green (G)**, **Yellow (Y)**, and **Blue (B)**.

At the start of each episode:

- The **taxi** begins at a random location on the grid.
- The **passenger** is randomly located at one of the four landmarks or inside the taxi.
- The **destination** is a random landmark (different from the passenger's starting location).

The agent's objective is to:

1. Navigate to the passenger's location.
2. Execute the "pickup" action.
3. Navigate to the destination.
4. Execute the "drop-off" action.

State Space

There are a total of **500 discrete states**, defined by the combination of:

- 25 taxi positions (5 rows × 5 columns),
- 5 possible passenger locations (4 fixed locations + inside the taxi),
- 4 possible destinations.

However, due to certain unreachable combinations (e.g., passenger already at destination), only **404 states are actually reachable** during episodes.

Action Space

The environment defines **6 primitive deterministic actions**:

- 0: Move South
- 1: Move North
- 2: Move East
- 3: Move West
- 4: Pickup
- 5: Drop-off

In the hierarchical setup, we also define **options**—macro-actions that guide the taxi toward a specific landmark (R, G, Y, or B) until the subgoal is achieved.

Rewards

The reward function is designed to encourage efficiency and penalize illegal or ineffective actions:

- **+20** for successfully delivering the passenger.
- **-1** per time step (encourages shorter paths).
- **-10** for attempting to pick up or drop off the passenger in an invalid state.

The **discount factor** for future rewards is set to $\gamma = 0.9$.

This structured yet simple environment provides an ideal testbed for exploring and comparing hierarchical reinforcement learning algorithms such as SMDP Q-learning and Intra-Option Q-learning.

1 Algorithm Descriptions

In this section, we discuss the two primary hierarchical reinforcement learning algorithms implemented in this assignment: **1-step SMDP Q-learning** and **Intra-Option Q-learning**. These algorithms are based on the framework introduced in Sutton, Precup, and Singh’s 1999 paper, which extends standard Markov Decision Processes (MDPs) to Semi-Markov Decision Processes (SMDPs) by incorporating temporally extended actions known as *options*.

1.1 1-step SMDP Q-Learning

1-step SMDP Q-learning is an extension of the traditional Q-learning algorithm to handle temporally extended actions. In this setting, each option $\mathcal{O}$ is defined by a triple $(\mathcal{I}, \pi, \beta)$, where:

- $\mathcal{I}$ is the initiation set of states from which the option can be executed.
- π is the intra-option policy that governs which primitive actions to take while the option is active.
- β is the termination condition, giving the probability of terminating the option in each state.

The agent selects an option from the set of available options based on the current state, follows its intra-option policy until it terminates, and then receives a cumulative reward R and time duration τ . The Q-value update rule is:

$$Q(s, o) \leftarrow Q(s, o) + \alpha \left[R + \gamma^\tau \max_{o'} Q(s', o') - Q(s, o) \right]$$

This allows the agent to evaluate and learn the value of entire option executions, rather than only individual primitive actions.

```

class TaxiOption:
    def __init__(self, name, initiation, termination):
        self.name = name
        self.initiation = initiation
        self.termination = termination
        self.q_table = None

[22] class HierarchicalAgent:
    def __init__(self, env, options, alpha=0.2, gamma=0.995, epsilon=0.3, epsilon_decay=0.9995):
        self.env = env
        self.options = options
        self.n_primitives = env.action_space.n
        self.n_actions = self.n_primitives + len(options)
        self.q_omega = np.zeros((env.observation_space.n, self.n_actions))
        self.alpha = alpha
        self.gamma = gamma
        self.epsilon = epsilon
        self.epsilon_decay = epsilon_decay
        self.reward_history = []

        for option in self.options:
            # option.q_table = np.zeros((env.observation_space.n, self.n_primitives))
            option.q_table = np.random.uniform(-0.01, 0.01,
                                                (env.observation_space.n, self.n_primitives))

    def decode_state(self, state):
        return self.env.unwrapped.decode(state)

    def get_action_mask(self, state):
        valid_actions = np.zeros(self.n_actions, dtype=bool)

        # Get valid primitives from environment dynamics
        valid_primitives = [a in self.env.unwrapped.P[state]
                            for a in range(self.n_primitives)]
        valid_actions[:self.n_primitives] = valid_primitives

        # Options
        for i, option in enumerate(self.options):
            valid_actions[self.n_primitives + i] = option.initiation(state)

        return valid_actions

    def option_policy(self, self, state, option):
        valid_actions = [action in self.env.unwrapped.P[state]
                        for action in range(self.n_primitives)]
        return np.argmax(option.q_table[state] * valid_actions)

```

Figure 1: Screenshot of hierarchicalAgent implementation here.

1.2 Intra-Option Q-Learning

Intra-option Q-learning is a more fine-grained approach where learning happens not only at option boundaries but also at each intermediate step. This algorithm updates the Q-values of all options whose policies could have taken the current action in the current state. The update rule is:

$$Q(s, o) \leftarrow Q(s, o) + \alpha [r + \gamma Q(s', o) - Q(s, o)]$$

This allows the agent to share learning across overlapping options and accelerates convergence by propagating information within options during their execution.

```

class SMDPLearner(HierarchicalAgent):
    def train(self, episodes=2000):
        for _ in tqdm(range(episodes)):
            state, _ = self.env.reset()
            done = False
            total_reward = 0

            while not done:
                action_mask = self.get_action_mask(state)

                if np.random.random() < self.epsilon:
                    action = np.random.choice(np.where(action_mask)[0])
                else:
                    action = np.argmax(self.q_omega[state] * action_mask)

                if action < self.n_primitives:
                    next_state, reward, terminated, truncated, _ = self.env.step(action)
                    done = terminated or truncated
                    total_reward += reward

                    next_mask = self.get_action_mask(next_state)
                    target = reward + self.gamma * np.max(self.q_omega[next_state][next_mask])
                    self.q_omega[state][action] += self.alpha * (target - self.q_omega[state][action])
                    state = next_state
                else:
                    option = self.options[action - self.n_primitives]
                    option_reward = 0
                    steps = 0
                    start_state = state

                    while not done and not option.termination(state):
                        action = self.option_policy(state, option)
                        next_state, reward, terminated, truncated, _ = self.env.step(action)
                        done = terminated or truncated

                        valid_actions = [a in self.env.unwrapped.P[state] for a in range(self.n_primitives)]
                        target = reward + self.gamma * np.max(option.q_table[next_state] * valid_actions)
                        option.q_table[state][action] += self.alpha * (target - option.q_table[state][action])

                        option_reward += (self.gamma ** steps) * reward
                        state = next_state
                        steps += 1

                    next_mask = self.get_action_mask(state)
                    target = option_reward + (self.gamma ** steps) * np.max(self.q_omega[state][next_mask])
                    self.q_omega[start_state][action] += self.alpha * (target - self.q_omega[start_state][action])
                    total_reward += option_reward

            self.reward_history.append(total_reward)
            self.epsilon = max(0.01, self.epsilon * self.epsilon_decay)

```

Figure 2: Screenshot of SMDP Q-learning implementation here

1.3 Option Definitions and Execution

In both algorithms, the core abstraction is the *option*, which serves as a macro-action. In this assignment, we define one option for each of the four target landmarks (Red, Green, Yellow, Blue), where the intra-option policy navigates the taxi to the designated location using primitive movements. Each option terminates once the taxi reaches the goal landmark.

The option set significantly reduces exploration complexity by enabling high-level behavior reuse. Furthermore, the separation of control into meta-policy (option selection) and intra-policy (action within option) forms the basis for scalable hierarchical learning.

```

class SMDPLearner(HierarchicalAgent):
    def train(self, episodes=2000):
        for _ in tqdm(range(episodes)):
            state, _ = self.env.reset()
            done = False
            total_reward = 0

            while not done:
                action_mask = self.get_action_mask(state)

                if np.random.random() < self.epsilon:
                    action = np.random.choice(np.where(action_mask)[0])
                else:
                    action = np.argmax(self.q_omega[state] * action_mask)

                if action < self.n_primitives:
                    next_state, reward, terminated, truncated, _ = self.env.step(action)
                    done = terminated or truncated
                    total_reward += reward

                    next_mask = self.get_action_mask(next_state)
                    target = reward + self.gamma * np.max(self.q_omega[next_state][next_mask])
                    self.q_omega[state][action] += self.alpha * (target - self.q_omega[state][action])
                    state = next_state
                else:
                    option = self.options[action - self.n_primitives]
                    option_reward = 0
                    steps = 0
                    start_state = state

                    while not done and not option.termination(state):
                        action = self.option_policy(state, option)
                        next_state, reward, terminated, truncated, _ = self.env.step(action)
                        done = terminated or truncated

                        valid_actions = [a in self.env.unwrapped.P[state] for a in range(self.n_primitives)]
                        target = reward + self.gamma * np.max(option.q_table[next_state] * valid_actions)
                        option.q_table[state][action] += self.alpha * (target - option.q_table[state][action])

                        option_reward += (self.gamma ** steps) * reward
                        state = next_state
                        steps += 1

                    next_mask = self.get_action_mask(state)
                    target = option_reward + (self.gamma ** steps) * np.max(self.q_omega[state][next_mask])
                    self.q_omega[start_state][action] += self.alpha * (target - self.q_omega[start_state][action])
                    total_reward += option_reward

            self.reward_history.append(total_reward)
            self.epsilon = max(0.01, self.epsilon * self.epsilon_decay)

```

Figure 3: Screenshot of Intra-Option Q-learning implementation here

2 SMDP vs. Intra-Option Learning: A Comparative Analysis

In this section, we present a comparative overview of the two hierarchical reinforcement learning approaches implemented in this assignment—**SMDP Q-Learning** and **Intra-Option Q-Learning**—from both conceptual and code perspectives.

2.1 Conceptual Differences

Aspect	SMDP Q-Learning	Intra-Option Learning
Temporal Scope	Treats options as atomic actions	Updates options at every timestep
Update Timing	Only updates when option terminates	Updates during option execution
Credit Assignment	Delayed reward propagation	Immediate reward propagation
Policy Hierarchy	Strict option hierarchy	Fluid option-primitive mixing

Table 1: Conceptual comparison between SMDP and Intra-Option Learning

2.2 Code Differences

1. Option Execution Flow

```

# SMDP (full option execution)
while not done and not option.termination(state):
    # Execute entire option before updates
    ...
# Single Q_omega update after option completion

# Intra-Option (step-wise execution)
if current_option: # Can switch options mid-execution
    action = option_policy()
else:
    action = primitive
# Updates happen every step

```

Listing 1: Option Execution Flow

2. Update Mechanism

```

# SMDP (delayed updates)
option_reward += (gamma ** steps) * reward # Discounted cumulative
self._update_q_omega(start_state, ...) # Single update at end

# Intra-Option (immediate updates)
for option in self.options: # Update all relevant options
    if option.initiation(state):
        option.q_table.update() # Update every step
self.q_omega.update() # Update every step

```

Listing 2: Q-Value Update Mechanism

3. Termination Handling

```

# SMDP (fixed commitment)
while not option.termination(state): # Must complete option
    ...

# Intra-Option (flexible)
if current_option.termination(state): # Can re-evaluate each step
    current_option = None

```

Listing 3: Termination Conditions

4. Action Selection

```

# SMDP (hierarchical)
Choose between primitives OR options
Once chosen, must complete option

# Intra-Option (hybrid)
Can mix primitive and option choices
Can interrupt options with new choices

```

Listing 4: Action Selection Strategy

2.3 Key Divergence Points in Code

- **SMDP** (`train()` function lines 12–35):
 - Uses a nested `while` loop to fully execute the selected option.
 - Accumulates discounted rewards over the duration of option execution.
 - Performs a single `Q_omega` update after the option completes.
- **Intra-Option** (`train()` function lines 15–35):
 - Uses a flat loop with option state tracking to allow dynamic option switching.
 - Updates all relevant options at each timestep.
 - Supports hybrid behavior of options and primitive actions.

2.4 When to Use Which

- **SMDP Q-Learning:** Suitable for long-duration behaviors where option commitment is desirable. Best when the environment has high-level subgoals that are clearly separable.
- **Intra-Option Q-Learning:** Useful for dynamic and fine-grained environments where flexibility in option execution provides an advantage. Often results in faster convergence but requires more computational updates.

These distinctions in implementation reflect their theoretical underpinnings: SMDP models options as temporal abstractions, whereas intra-option learning allows continuous adaptation through overlapping action and option policies.

3 Results and Visualization

To evaluate the performance and learning dynamics of the SMDP and Intra-Option Q-learning algorithms, we analyze two key metrics:

- The learned Q-values for the meta-controller (Q_ω) and individual options.
- Episode reward curves indicating convergence behavior over training episodes.

3.1 Learned Q_ω Values

The Q_ω values represent the meta-controller’s expected return for executing a given option from a particular state. Below are the visualizations of learned Q_ω values for both methods:

Meta-Policy (Q-Omega) Values

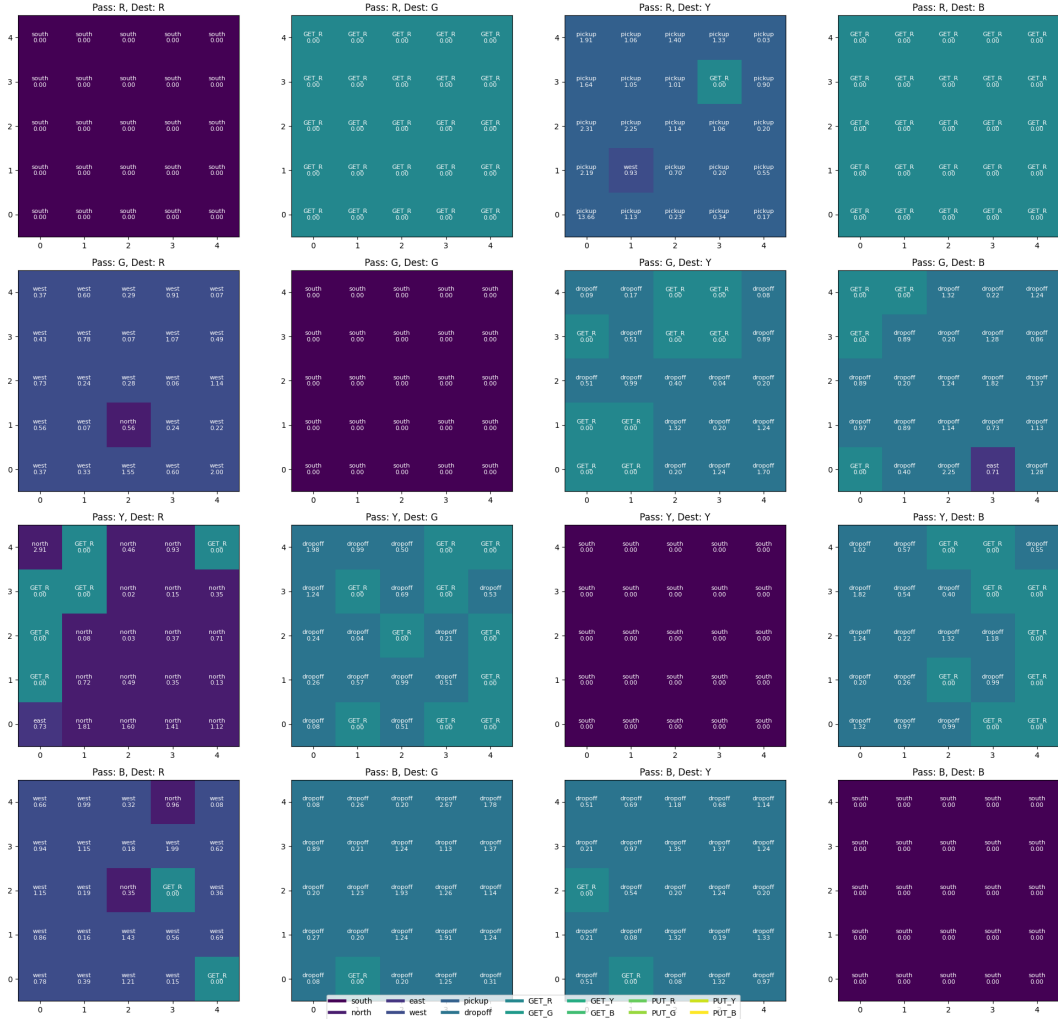


Figure 4: Q_ω values learned via SMDP Q-learning.

Meta-Policy (Q-Omega) Values

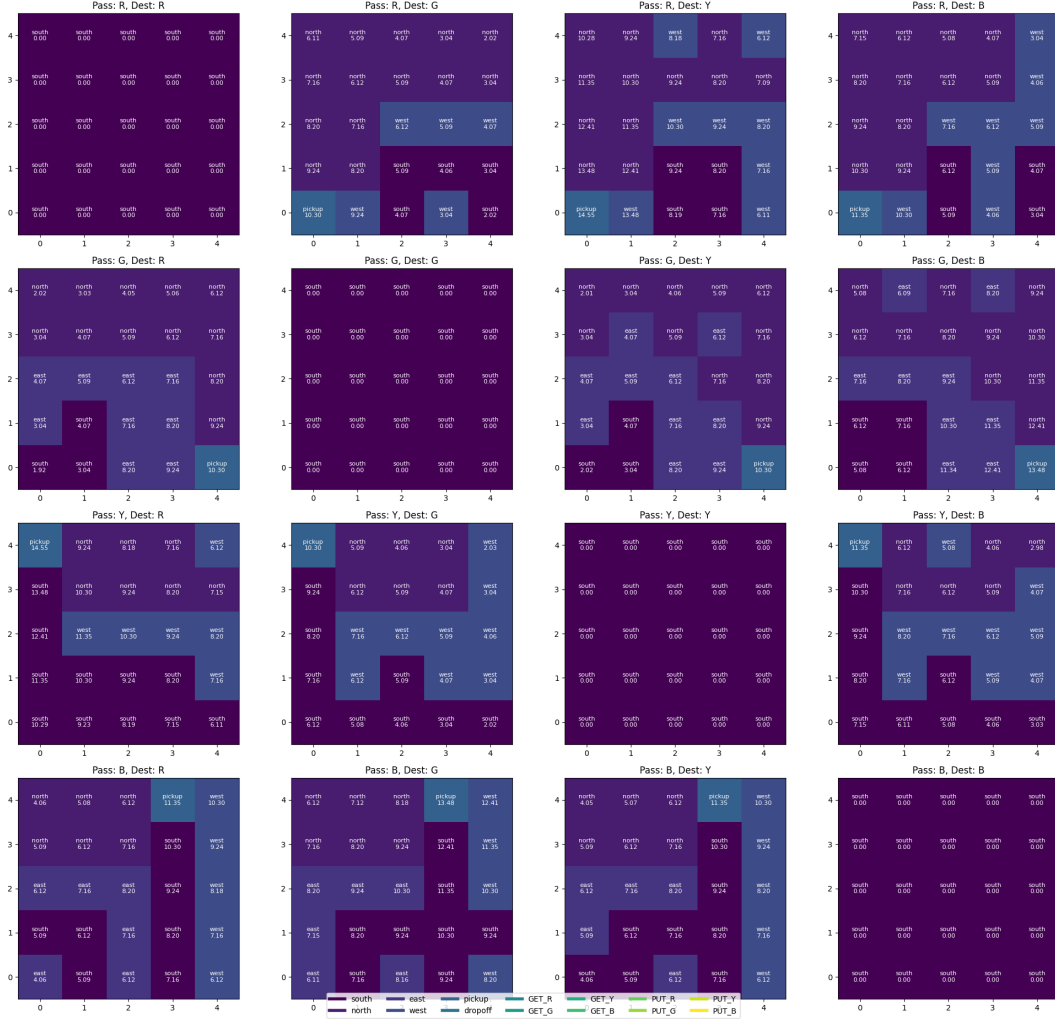


Figure 5: Q_ω values learned via Intra-Option Q-learning.

3.2 Option Value Estimations

The following plots show the evolution of option-level Q-values. These represent how the value function of each option evolves during training based on their internal policies.

Policy for all options



Figure 6: Option value estimations – SMDP Q-learning.

Policy for all options



Figure 7: Option value estimations – Intra-Option Q-learning.

3.3 Reward Curves

The cumulative reward per episode is plotted for both approaches over the training period. This provides insight into sample efficiency and learning stability.

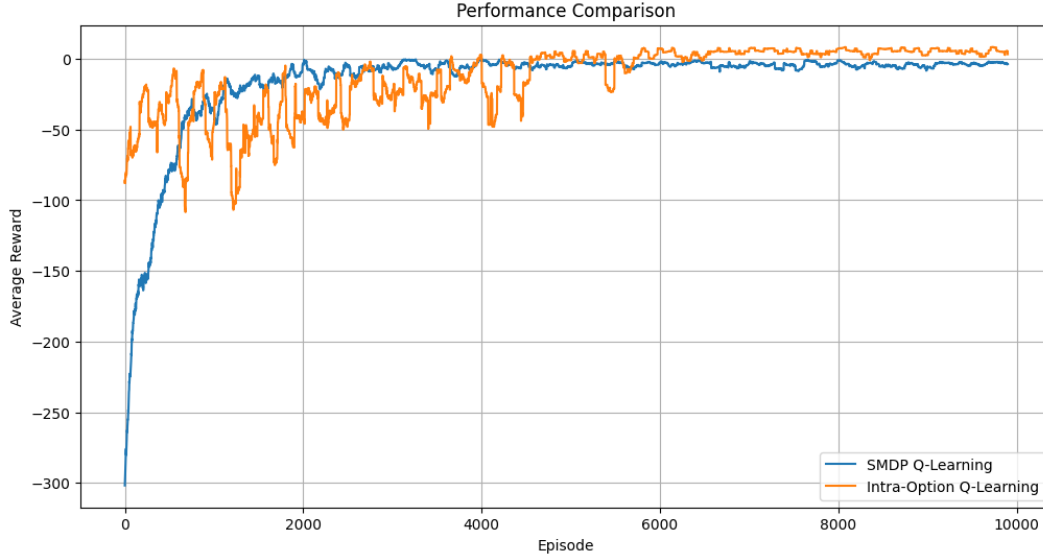


Figure 8: Reward curve: SMDP vs. Intra-Option Q-learning.

Observations

- Intra-Option learning converges faster due to more frequent updates and finer credit assignment.
- SMDP exhibits smoother learning but with delayed convergence, owing to option commitment.
- Policy learned by the SMDP tends to prioritize primitive actions more compared to Intra option. It was observed from the visualization of Q Values, we see that if we go greedy with the Q values, mostly primitive action are chosen in case of SMDP.
- The Above phenomena can be attributed to the long duration of options, and strict termination conditions. If we allow the options to be terminated early (also called as interrupted options), advantage of the options can be enhanced

4 Exploring an Alternate Set of Options

In this section, we explore whether it is possible to solve the taxi domain using a different set of options that are mutually exclusive from the original navigation-based options (which moved the taxi to fixed landmarks like Red, Blue, etc.).

Initially, we also tried to implement one-step SMDP Q learning with options whose policies were hard-coded. Also, we introduce learning policies for each option. We observed that initially, average reward increases with training; however, in the later stages of training, we were able to achieve the best performance for options which we hard-coded with optimal policies. Whereas when we tried to learn the option policies as well, the agent could not achieve the same performance, this could be attributed to sparse updates compared to a large number of parameters.

4.1 SMDP Q-Learning with Hardcoded Options

We implemented and ran the SMDP Q-learning algorithm using this alternate set of options. Below, we present three key visualizations that capture its learning performance.

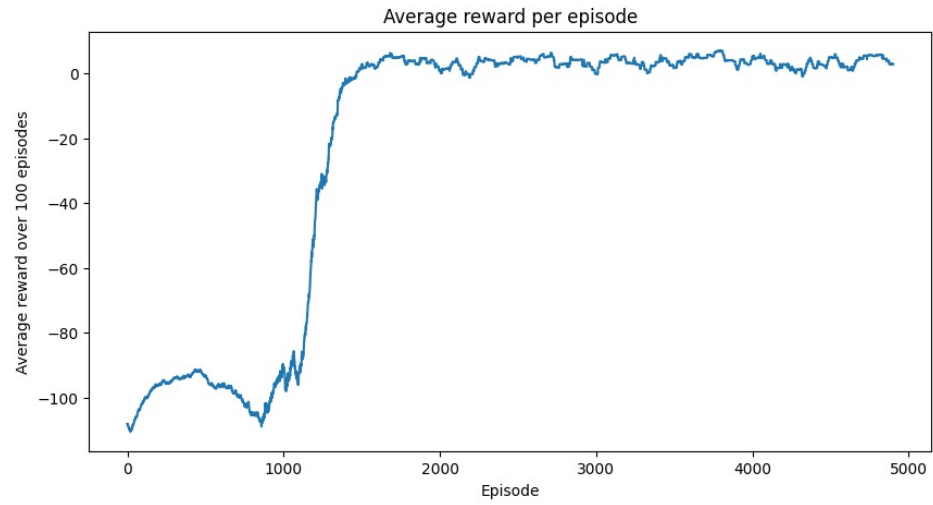
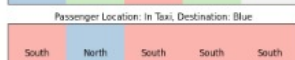
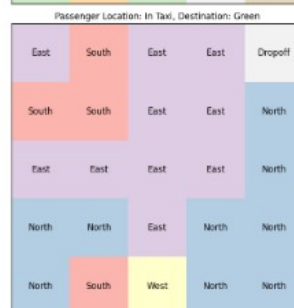
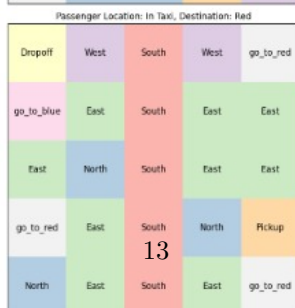
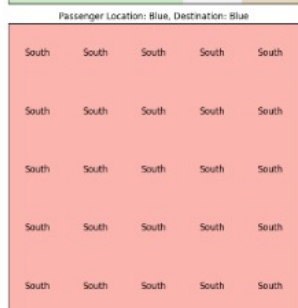
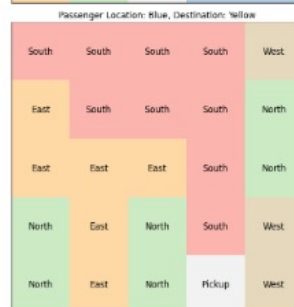
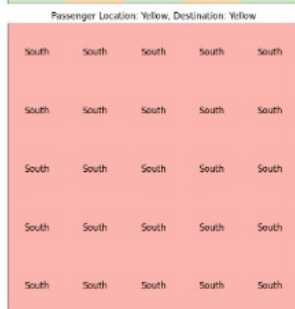
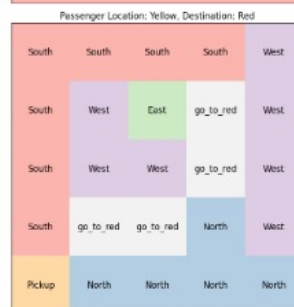
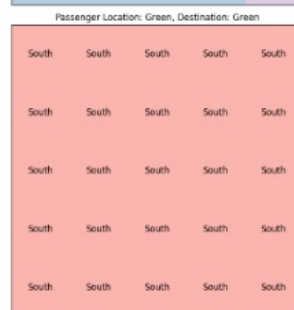
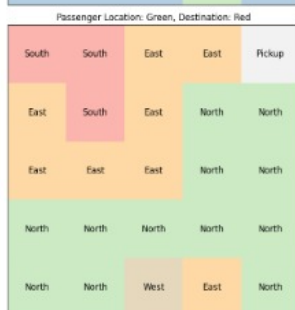
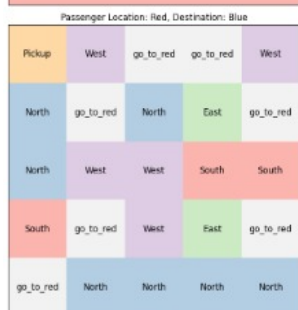
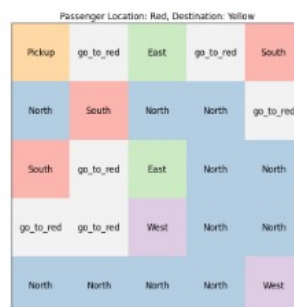
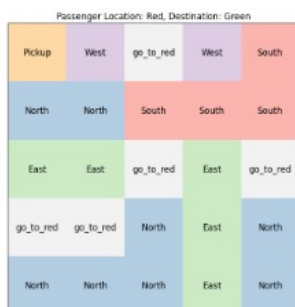
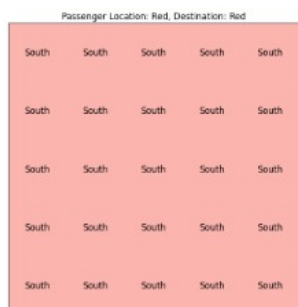


Figure 9: Reward curve using hardcoded options with SMDP Q-learning.



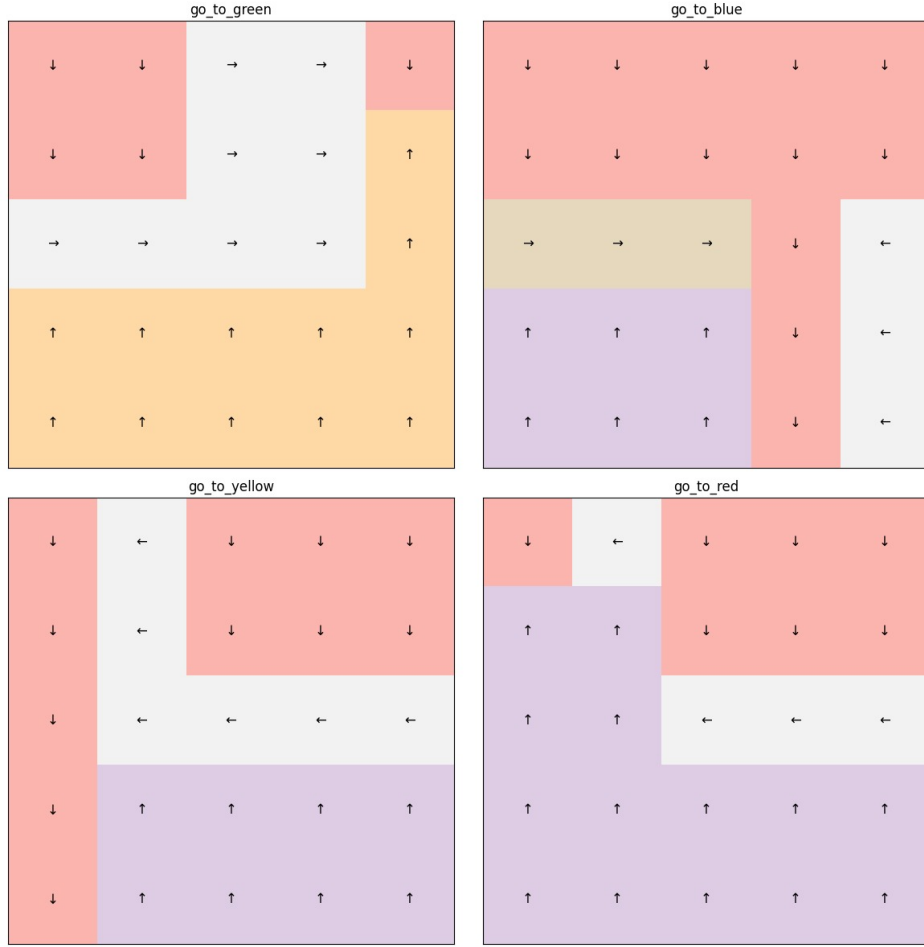


Figure 11: Learned option policies (hardcoded options).

4.2 Conclusion

In our experiments, We have tried three different ways of defining options, one, where we hard-coded the option policies. Second, we are learning the option policies as well. We have also experimented with different set of options, whose results are reported above. We have observed that choice of options, length of options and the provision of interrupted options are crucial for learning